



Basic Information:

Experiment No:03

Date:17/09/2026

Title: UPI Payment and Digital Wallet System using Interface and Exception Handling



Objective/Aim:

To design and implement in Java using, simulating real-world digital payment operations such as wallet creation, money addition, UPI payments, balance checking, and transaction validation.



System Requirements:

Hardware:

- > Standard PC/Laptop
- > Intel Core i3/i5 processor or equivalent
- > Minimum 4 GB RAM
- > Minimum 500 MB free storage

Software:

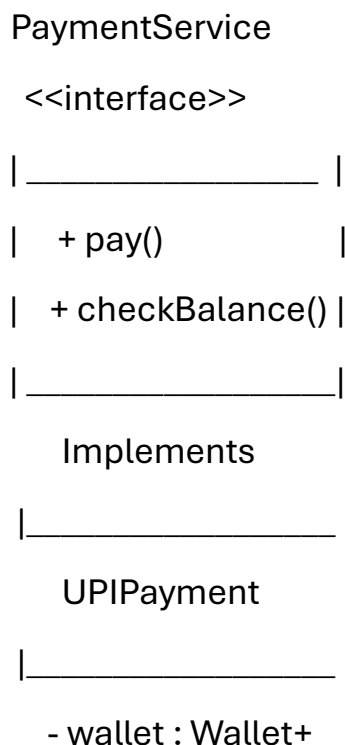
- > Operating System: Windows / Linux / macOS
- > Java Development Kit (JDK 17 or JDK 21)
- > Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- > Command Prompt / Terminal

Algorithm/Logic Description:

1. Create an interface named PaymentService.
2. Declare pay() and checkBalance() methods in the interface.
3. Create a Wallet class.
4. Declare user information and balance as private data members.
5. Create a constructor to initialize the wallet details.

6. Define addMoney() to add money to the wallet.
7. Create an UPIPayment class that implements PaymentService.
8. Create an object of Wallet and associate it with the UPIPayment object.
9. Validate the UPI ID using String operations.
10. If the UPI ID is invalid, throw InvalidUPIException.
11. Validate the payment amount.
12. If the amount is zero or negative, throw InvalidAmountException.
13. Check whether sufficient balance is available.
14. If the balance is insufficient, throw InsufficientBalanceException.
15. If all validations are successful, deduct the payment amount from the wallet balance.
16. Display the payment success message.
17. Use try-catch-finally to handle transaction exceptions.
18. Display the final wallet balance and wallet details.

UML Class Diagram:



+pay()
+ checkBalance()

Uses

Wallet

- userName
- mobileNumber
- upild
- balance

+ addMoney()
+ getBalance()
+ deductMoney()
+ displayWalletDetails()

UPI

<<main class>>

+ main()
creates objects

/ \

Wallet UPIPayment

Source Code:

```
package myprgms;
```

```
import java.util.Scanner;

// Interface

interface PaymentService {

    void pay(String upild, double amount)

        throws InvalidUPIException, InvalidAmountException,

            InsufficientBalanceException;

    void checkBalance();

}

// Custom Exception 1

class InsufficientBalanceException extends Exception {

    InsufficientBalanceException(String message) {

        super(message);

    }

}

// Custom Exception 2

class InvalidUPIException extends Exception {

    InvalidUPIException(String message) {

        super(message);

    }

}

// Custom Exception 3

class InvalidAmountException extends Exception {

    InvalidAmountException(String message) {

        super(message);

    }

}
```

```
// Wallet Class

class Wallet {

private String userName;

private String mobileNumber;

private String upild;

private double balance;

// Constructor

Wallet(String userName, String mobileNumber,

String upild, double balance) {

this.userName = userName;

this.mobileNumber = mobileNumber;

this.upild = upild;

this.balance = balance;

}

// Add money to wallet

void addMoney(double amount) {

if (amount > 0) {

balance = balance + amount;

System.out.println("Money added successfully: Rs." + amount);

}

else {

System.out.println("Invalid amount.");

}

}

// Get wallet balance

double getBalance()
```

```

return balance;

    }

// Deduct payment amount

void deductMoney(double amount) {

    balance = balance - amount;

    }

// Display wallet details

void displayWalletDetails() {

    System.out.println("\n----- Wallet Details -----");

    System.out.println("User Name   : " + userName);

    System.out.println("Mobile Number : " + mobileNumber);

    System.out.println("UPI ID      : " + upild);

    System.out.println("Balance    : Rs." + balance);

    }

}

// UPI Payment Class

class UPIPayment implements PaymentService {

private Wallet wallet;

// Constructor

UPIPayment(Wallet wallet) {

    this.wallet = wallet;

    }

// Pay method

public void pay(String upild, double amount)

throws InvalidUPIException,

        InvalidAmountException {

```

```

InsufficientBalanceException {
    // Validate UPI ID
    if (!upild.contains("@")) {
        throw new InvalidUPIException(
            "Invalid UPI ID. UPI ID must contain '@.'");
        }
    // Validate amount
    if (amount <= 0) {
        throw new InvalidAmountException("Invalid amount. Amount must be
greater than zero.");
        }
    // Check balance
    if (amount > wallet.getBalance()) {
        throw new InsufficientBalanceException("Insufficient balance. Payment
cannot be processed.");
        }
    // Deduct amount
    wallet.deductMoney(amount)
    System.out.println("\nPayment Successful!");
    System.out.println("Paid To    : " + upild);
    System.out.println("Amount Paid : Rs." + amount);
    }
    // Check balance
    public void checkBalance() {
        System.out.println("Available Balance: Rs."+ wallet.getBalance());
        }
    }
}

```

```
// Main Class

public class UPI {

public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

// Create wallet

Wallet wallet = new Wallet("Sai","9876543210", "sai@upi",5000);

// Create UPI payment object

UPIPayment payment = new UPIPayment(wallet)

wallet.displayWalletDetails();

// Add money

System.out.print("\nEnter amount to add: ");

double addAmount = sc.nextDouble();

wallet.addMoney(addAmount);

// Payment details

System.out.print("\nEnter receiver UPI ID: ");

String receiverUPI = sc.next();

System.out.print("Enter payment amount: ");

double paymentAmount = sc.nextDouble();

// Exception handling

    try {

        payment.pay(receiverUPI, paymentAmount);

    } catch (InvalidUPIException e) {

        System.out.println("Transaction Failed: " + e.getMessage());

    } catch (InvalidAmountException e) {

        System.out.println("Transaction Failed: + e.getMessage());

    } catch (InsufficientBalanceException e) {
```

```

System.out.println("Transaction Failed: "+ e.getMessage());
} finally{
System.out.println("\nTransaction processing completed.");

    }

// Display final balance

System.out.println("\n----- Final Balance -----");

payment.checkBalance();

// Display final wallet details

wallet.displayWalletDetails();

sc.close();

}

}

```

Output Screenshot:

```

----- Wallet Details -----
User Name : sai
Mobile Number : 9876543210
UPI ID : sai@upi
Balance : Rs.5000.0

Enter amount to add: 2000
Money added successfully: Rs.2000.0

Enter receiver UPI ID: ravi@upi
Enter payment amount: 1000

Payment Successful!
Paid To : ravi@upi
Amount Paid : Rs.1000.0

Transaction processing completed.

----- Final Balance -----
Available Balance: Rs.6000.0

----- Wallet Details -----
User Name : sai
Mobile Number : 9876543210
UPI ID : sai@upi
Balance : Rs.6000.0

```

Result:

The UPI Payment System was successfully implemented in Java using interfaces, classes, and custom exceptions. The program allows users to add money, make UPI payments, and check the wallet balance. It also validates the UPI ID, payment amount, and available balance. Custom exceptions are used to handle invalid UPI IDs, invalid amounts, and insufficient balance. Thus, the program successfully demonstrates a basic UPI payment system with exception handling.